Kurs: Linux + GitHub Actions, Teil 1 (v4)

Vorbereitung

Für diesen Kurs brauchst du ein GitHub-Konto und ein eigenes Lern-Repository mit aktivierten GitHub Actions. Du brauchst Schreibzugriff, um Änderungen dorthin zu pushen und Workflows manuell zu starten. Für die Lern-Repositorys in diesem Kurs brauchst du keinen Git Flow. Lege Dateien anhand der Codeblöcke im Kurstext an; die Vorlagen liegen auch im Kurs-Repository. Wenn du YAML oder Git erst kennenlernen möchtest, findest du die Grundlagen in YAML-Grundlagen und Git-Grundlagen.

1. Lege auf GitHub ein öffentliches Repository `linux-lernen` mit README an.
2. Erstelle Dateien im Browser über **Add file → Create new file**. Gibst du als Dateinamen einen Pfad wie `daten/einkauf.txt` ein, legt GitHub die Ordner automatisch an. Leere Ordner lassen sich im Browser nicht anlegen. Klicke zum Speichern auf **Commit changes**; dadurch wird direkt auf dem ausgewählten Branch committed.
3. Committe die Workflow-Dateien auf den **Standardbranch** (meist `main`). Damit `workflow_dispatch` ausgelöst werden kann, muss die Workflow-Datei dort liegen. Beim manuellen Start kannst du anschließend einen Branch auswählen.

Einen Arbeitsbranch über die GitHub-Oberfläche erstellen: Öffne die Hauptseite deines Repositorys und klicke in der Dateiansicht auf die Branch-Auswahl. Gib im Feld **Find or create a branch...** einen Namen ein, zum Beispiel `aufgabe-skript`, und wähle **Create branch: aufgabe-skript**. Falls nötig, wähle vorher den Standardbranch als Ausgangspunkt. Arbeite bei den Aufgaben, die einen Arbeitsbranch verlangen, in diesem Branch weiter.[1] Wenn du die Dateien im GitHub-Browser bearbeitest, speichert **Commit changes** direkt auf dem ausgewählten Branch. Das ersetzt den in den Aufgaben genannten Push; bei Arbeit mit Git auf deinem Rechner musst du weiterhin pushen.

Die Lizenzdateien bleiben im Kurs-Repository. Lizenzhinweise in den Code- und Beispieldatenblöcken verweisen auf diese Dateien; kopiere sie nicht in dein Lern-Repository.

> Workflow-Dateien müssen im Hauptverzeichnis des Projekts unter `.github/workflows/` liegen und auf `.yml` oder `.yaml` enden. Die gezeigten Fehlerproben sind beabsichtigt. Wo nötig, sind sie so eingerichtet, dass ein Fehler gezeigt wird, ohne den gesamten Übungslauf abzubrechen.

---

[1] Quelle zum Erstellen von Branches: https://docs.github.com/en/pull-requests/how-tos/commit-changes/managing-branches-within-your-repository[2] (geprüft am 06.11.2026).

Lektion 1: Hallo Linux

Lernziel: Du kannst einen einfachen Workflow mit `workflow_dispatch`, einem Job und aufeinanderfolgenden Schritten lesen und die Ausgaben grundlegender Linux-Befehle `echo`, `whoami`, `pwd`, `date`, `uname` im Runner finden.

Datei anlegen: `.github/workflows/01-hallo-linux.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 01 - Hallo Linux

on:
  workflow_dispatch:

jobs:
  erkunden:
    runs-on: ubuntu-latest
    steps:
      - name: Begrüßung
        run: echo "Hallo aus dem Linux-Runner!"

      - name: Wer bin ich?
        run: whoami

      - name: Wo bin ich?
        run: pwd

      - name: Welches Datum ist heute?
        run: date

      - name: Welches System läuft hier?
        run: uname -a
```

Die Workflow-Dateien liegen unter `.github/workflows/` und enden auf `.yml`. Unser Beispiel oben startet ausschließlich manuell über `workflow_dispatch`. Damit GitHub **Run workflow** anbietet, muss die Workflow-Datei auf dem **Standardbranch deines Lern-Repositorys** liegen (meist `main`). Beim Start wählst du den Branch, dessen Dateien ausgeführt werden sollen.

Diese Hinweise entsprechen der GitHub-Dokumentation zu manuellen Workflow-Starts und zum Ereignis `workflow_dispatch`.[3]

So startest und prüfst du einen Workflow: Öffne den Tab **Actions**, wähle links den genannten Workflow und klicke **Run workflow** auf der rechten Seite. Wähle bei Bedarf den Branch aus und starte den Lauf. Öffne den Lauf und danach den Job; in den Schritten siehst du Befehle und Ausgaben. Prüfe das Ergebnis anhand der in der Lektion genannten Kontrollpunkte.

Die Workflows verwenden `ubuntu-latest`, einen von GitHub bereitgestellten Ubuntu-Runner. Die genaue Ubuntu-Version kann sich ändern. Die vollständigen Workflow-Dateien und Skriptvorlagen liegen zusätzlich im Kurs-Repository.

Ausprobieren

1. Öffne im Tab **Actions** den Workflow **„01 - Hallo Linux“**.
2. Klicke **Run workflow**, wähle den Branch für den Lauf und starte ihn.
3. Öffne den Lauf und den Job `erkunden`.
4. Klappe jeden Schritt auf. Prüfe, dass die Begrüßung erscheint und `whoami`, `pwd`, `date` und `uname` jeweils eine Ausgabe liefern.

Kontrolle: `pwd` zeigt das aktuelle Arbeitsverzeichnis des Runners. Datum, Benutzername und Systemausgabe hängen von der Runner-Umgebung und dem Zeitpunkt ab.

Aufbau erklärt

Zeile	Bedeutung
`name`	Anzeigename des Workflows
`on: workflow_dispatch`	Erlaubt manuellen Start, wenn die Datei auf dem Standardbranch liegt
`jobs`	Liste der Aufgaben
`runs-on: ubuntu-latest`	Läuft auf der von GitHub aktuell bereitgestellten Ubuntu-Umgebung. Die genaue Version kann sich mit der Zeit ändern.
`steps`	Schritte, die nacheinander laufen

[3] GitHub dokumentiert, dass die Workflow-Datei für `workflow_dispatch` auf dem Standardbranch liegen muss. Quellen: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[4] und https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#workflow_dispatch[5] (geprüft am 06.11.2026).

Zeile	Bedeutung
`run`	Ein Linux-Kommando

Kommandos erklärt

Kommando	Zweck
`echo "text"`	Text ausgeben
`whoami`	Aktueller Benutzername (auf GitHub-Runnern üblicherweise `runner`)
`pwd`	Aktuelles Verzeichnis (**p**rint **w**orking **d**irectory)
`date`	Datum und Uhrzeit
`uname -a`	Systeminfos (`-a` = all). Die Ausgabe hängt von der Runner-Umgebung ab.

Aufgaben

1. Füge einen Schritt hinzu, der `echo "Mein Name ist <dein Name>"` ausgibt.
2. Gib mit `date +%Y-%m-%d` nur das aktuelle Datum aus. Die Ausgabe sieht zum Beispiel so aus: `2026-10-19`.
3. Finde mit `uname -r` die Kernel-Version heraus.

Lektion 2: Dateien und Ordner

Lernziele: Du kannst Dateien im Runner-Verzeichnis auflisten und lesen, Dateien und Ordner anlegen, kopieren, umbenennen und löschen. Du verstehst, warum der Workflow `checkout` benötigt.

In den Workflows 2 bis 4 lädt `actions/checkout@v7`[6] die Dateien aus dem ausgeführten Branch in den Arbeitsbereich des Jobs.

Datei anlegen: Lege `daten/einkauf.txt` in deinem Lern-Repository an. Die zwei Kommentarzeilen nennen Herkunft und Lizenz; die Einkaufsliste enthält fünf Einträge:

```
# Linux CI – Beispieldaten
# Lizenz: CC BY 4.0 (siehe LICENSE-CONTENT.txt im Kurs-Repository)
Brot
Milch
Käse
Äpfel
Kaffee
```

Datei anlegen: `.github/workflows/02-dateien.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 02 - Dateien und Ordner

on:
  workflow_dispatch:

jobs:
  dateien:
    runs-on: ubuntu-latest
    steps:
      - name: Repo auschecken
        uses: actions/checkout@v7

      - name: Was liegt im Repo?
        run: ls -la

      - name: Inhalt von daten/ anzeigen
        run: ls -l daten
```

[6] `actions/checkout@v7` verwendet Node.js 24. Am 06.11.2026 ist v7.0.1 die aktuelle Patch-Version; das Major-Tag `@v7` folgt der jeweils neuesten Patch-Version und pinnt sie nicht exakt. Quelle: https://github.com/actions/checkout/releases[7] (geprüft am 06.11.2026).

```
- name: Einkaufsliste lesen
  run: cat daten/einkauf.txt

- name: Ordner und Dateien anlegen
  run: |
    mkdir -p arbeit
    touch arbeit/notizen.txt
    echo "Erste Notiz" > arbeit/notizen.txt
    ls -l arbeit

- name: Kopieren und umbenennen
  run: |
    cp daten/einkauf.txt arbeit/kopie.txt
    mv arbeit/kopie.txt arbeit/liste.txt
    ls arbeit

- name: Temporäre Datei löschen
  run: |
    rm arbeit/notizen.txt
    echo "--- nach dem Löschen ---"
    ls arbeit

- name: Wiederholtes mkdir ohne -p testen
  continue-on-error: true
  run: |
    mkdir testordner
    mkdir testordner
    echo "Dieser Befehl wird nach dem Fehler nicht ausgeführt"
```

Warum `checkout`?

Jeder Job läuft auf einer eigenen Runner-Umgebung. Damit die Dateien **deines Repositorys** dort bereitliegen, lädt `actions/checkout@v7` sie in den Arbeitsbereich. Die Schritte **innerhalb desselben Jobs** verwenden denselben Arbeitsbereich: Was ein Schritt anlegt, ist im nächsten Schritt noch da. Bei GitHub-gehosteten Runnern stellt GitHub für **jeden Job** eine **eigene Umgebung** bereit.

`uses:` ruft eine fertige Action auf; `run:` führt Befehle in der Shell aus. Der Test mit doppeltem

`mkdir` ist ein eigener Schritt. `continue-on-error: true` lässt den Workflow nach dem erwarteten Fehler fortsetzen; der Schritt kann als fehlgeschlagen angezeigt werden, ohne den gesamten Job fehlschlagen zu lassen.

Kommandos erklärt

Kommando	Zweck
`ls`	Dateien auflisten
`ls -l`	Mit Details wie Rechten, Größe und Datum
`ls -a`	Auch versteckte Dateien anzeigen (sie beginnen mit `.`)
`cat datei`	Dateiinhalt ausgeben
`mkdir ordner`	Ordner anlegen
`mkdir -p ordner`	Ordner anlegen, ohne Fehler, wenn er schon existiert
`touch datei`	Datei anlegen, falls sie fehlt; sonst Zeitstempel aktualisieren
`cp quelle ziel`	Kopieren
`mv quelle ziel`	Verschieben oder umbenennen
`rm datei`	Ohne Papierkorb löschen; eine Wiederherstellung ist nicht garantiert
`>`	Ausgabe in eine Datei schreiben; überschreibt den bisherigen Inhalt
`>>`	Ausgabe an eine Datei anhängen

Ausprobieren und kontrollieren

1. Starte **„02 - Dateien und Ordner“** über **Actions → Run workflow**.
2. Öffne den Job `dateien`. Prüfe in der Ausgabe, dass die Einkaufsliste fünf Zeilen enthält und `arbeit/liste.txt` nach dem Kopieren und Umbenennen angezeigt wird.
3. Im letzten Schritt wird `testordner` zweimal angelegt. Der zweite Aufruf meldet, dass der Ordner bereits existiert; die anschließende `echo`-Zeile wird nicht ausgeführt. Der Workflow läuft trotzdem weiter, weil dieser einzelne Fehler als erwartet behandelt wird.

Aufgaben

1. Ergänze im Schritt „Kopieren und umbenennen“ die Befehle `mkdir -p backup` und `cp daten/einkauf.txt backup/einkauf.txt`. Prüfe mit `ls backup`, ob die Kopie vorhanden ist.
2. Hänge mit `echo "Butter" >> arbeit/liste.txt` eine Zeile an und zeige die Datei mit `cat`. Prüfe, dass danach sechs Zeilen erscheinen.
3. Ändere im letzten Schritt beide Vorkommen von `testordner` zu `testordner-p` und ergänze `-p` bei beiden `mkdir`-Befehlen. Führe den Workflow erneut aus: Beide Befehle sollen ohne Fehlermeldung funktionieren.

> `rm` löscht im Workflow nur die dort erzeugte temporäre Datei im Arbeitsverzeichnis des Jobs. Auf deinem eigenen Rechner löscht `rm` ohne Papierkorb; Wiederherstellung ist nicht garantiert. Prüfe Pfad und Dateinamen, bevor du den Befehl ausführst.

Lektion 3: Texte durchsuchen und auswerten

Lernziele: Du kannst Anfang und Ende einer Datei anzeigen, Zeilen zählen, passende Zeilen suchen und Befehle mit Pipes verbinden. Du übst `head`, `tail`, `wc`, `grep`, `sort`, `cut` und die Pipe (`|`).

Vorbereitung: Lege `daten/server.log` an. Die Zeitstempel sind frei erfundene Beispieldaten und müssen nicht zum aktuellen Datum passen.

```
# Linux CI – Beispieldaten
# Lizenz: CC BY 4.0 (siehe LICENSE-CONTENT.txt im Kurs-Repository)
2026-10-19 08:01:12 INFO Server gestartet
2026-10-19 08:05:33 INFO Benutzer anna angemeldet
2026-10-19 08:07:45 ERROR Datenbank nicht erreichbar
2026-10-19 08:09:02 INFO Benutzer ben angemeldet
2026-10-19 08:12:10 WARNING Speicher bei 85 Prozent
2026-10-19 08:15:27 ERROR Zeitüberschreitung bei Anfrage
2026-10-19 08:20:41 INFO Benutzer anna abgemeldet
2026-10-19 08:25:55 ERROR Datenbank nicht erreichbar
2026-10-19 08:30:00 INFO Backup gestartet
```

Datei anlegen: `.github/workflows/03-texte.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 03 - Texte auswerten

on:
  workflow_dispatch:
    inputs:
      fehlerfall:
        description: "FATAL-Suche ohne Treffer ausführen"
        required: true
        type: boolean
        default: false

jobs:
  logs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Anfang und Ende
        run: |
          echo "--- erste 3 Zeilen ---"
          head -n 3 daten/server.log
          echo "--- letzte 2 Zeilen ---"
          tail -n 2 daten/server.log

      - name: Zeilen zählen
        run: wc -l daten/server.log

      - name: Alle Fehler finden
        run: grep "ERROR" daten/server.log

      - name: Fehler zählen
        run: grep -c "ERROR" daten/server.log

      - name: Pipe - Fehler sortieren
        run: grep "ERROR" daten/server.log | sort
```

```
      - name: Pipe - Nur die Meldungen der Fehler
        run: grep "ERROR" daten/server.log | cut -d' ' -f4-

      - name: FATAL-Suche ohne Treffer (absichtlicher Fehler)
        if: ${{ inputs.fehlerfall }}
        continue-on-error: true
        run: grep -c "FATAL" daten/server.log
```

Das Pipe-Prinzip

```
grep "ERROR" server.log  |  sort  |  head -n 1
   ↓ liefert Zeilen         ↓ sortiert  ↓ nimmt die erste
```

Die Ausgabe des linken Kommandos wird zur Eingabe des rechten. Achtung: Dieses Beispiel liefert die **alphabetisch erste** Fehlerzeile, nicht die zeitlich letzte. Das ist wichtig für die Knobelaufgabe unten.

Kommandos erklärt

Kommando	Zweck
`head -n 3`	Erste 3 Zeilen
`tail -n 2`	Letzte 2 Zeilen
`wc -l`	Zeilen zählen (**w**ord **c**ount, `-l` = lines)
`grep "text" datei`	Zeilen mit dem Suchtext anzeigen
`grep -c`	Nur die Anzahl der Treffer ausgeben
`grep -i`	Groß- und Kleinschreibung ignorieren
`sort`	Zeilen sortieren
`cut -d' ' -f4-`	Felder ab Feld 4 ausschneiden; ein einzelnes Leerzeichen ist der Trenner

Pipe (`|`): Die Ausgabe des linken Kommandos wird zur Eingabe des rechten Kommandos weitergereicht. Zum Beispiel sucht `grep -E "a|b"` nach `a` **oder** `b`.

> **Fehlerprobe: `grep` ohne Treffer.** Beim manuellen Start von **„03 - Texte auswerten“** kannst du die Eingabe `fehlerfall` auf `true` setzen. In der Actions-Oberfläche wird sie mit der Beschreibung `FATAL-Suche ohne Treffer ausführen` angezeigt. `grep -c "FATAL" daten/server.log` gibt `0` aus, aber liefert Exit-Code 1, weil nichts gefunden wurde. Der Schritt wird als fehlgeschlagen angezeigt; `continue-on-error: true` sorgt dafür, dass der Job trotzdem erfolgreich enden kann. Ohne diese Einstellung wäre dies der letzte Schritt und der Job würde fehlschlagen.
>
> `grep` verwendet Exit-Code 2 für einen Fehler wie eine nicht lesbare oder fehlende Datei. `|| true` würde sowohl „keine Treffer“ als auch solche echten Fehler abfangen; setze es daher nur ein, wenn diese Unterscheidung unwichtig ist.
>
> *Für Neugierige:* Auf Linux führt GitHub Actions `run:` ohne explizite Shell-Angabe standardmäßig mit Bash und `-e` aus (mit Fallback auf `sh -e`, falls Bash fehlt). Bei ausdrücklich gesetztem `shell: bash` kommt `-o pipefail` hinzu. Ohne `pipefail` zählt bei einer Pipe normalerweise der Status des letzten Befehls; mit `pipefail` kann auch ein fehlgeschlagener früherer Befehl den Pipe-Status fehlschlagen lassen.

Ergebniskontrolle

- `wc -l daten/server.log` zählt 11 Zeilen: zwei Lizenzhinweis-Zeilen und neun Logeinträge.
- `grep -c "ERROR" daten/server.log` gibt `3` aus; die Lizenzhinweise enthalten keine Schweregrade.
- Die Sortierung in einer Pipe sortiert alphabetisch; sie ändert nicht die Reihenfolge der Originaldatei.
- Das Feld `-f4-` zeigt bei diesem Beispiel die Meldung nach Datum, Uhrzeit und Schweregrad. `cut -d' '` trennt an einzelnen Leerzeichen und ist nicht für beliebig viele Leerzeichen gedacht.

Aufgaben

1. Wie viele `INFO`-Zeilen gibt es? Prüfe mit `grep -c "INFO" daten/server.log`.
2. Welche Zeilen enthalten das Wort `anna`? Verwende `grep` und vergleiche die Ausgabe mit dem Log.
3. Zeige nur die Warnungen und Fehler an: `grep -E "WARNING|ERROR" daten/server.log`. Zähle die ausgegebenen Zeilen.
4. Starte den Workflow erneut und setze beim manuellen Start die Eingabe `fehlerfall` (Beschreibung: `FATAL-Suche ohne Treffer ausführen`) auf `true`. Beobachte Ausgabe und Status des letzten Schritts. Warum gibt `grep -c` eine Null aus und meldet trotzdem einen Fehlerstatus? Der genaue Schrittstatus in der GitHub-Oberfläche ist noch nicht geprüft.

5. Knobelaufgabe: Zeige die **zeitlich letzte** Fehlermeldung an (`grep` + `tail`). Prüfe, ob das Log chronologisch geordnet ist.

Lektion 4: Variablen und das erste Shell-Skript

Lernziele: Du kannst Shell-Variablen und Workflow-Eingaben unterscheiden, ein Bash-Skript mit einem Parameter ausführen und erklären, warum Eingaben über eine Umgebungsvariable übergeben werden. Das Kommando `chmod` anwenden.

Datei anlegen: `skripte/begruessung.sh`. Der Ordner `skripte/` entsteht über den Pfad im Dateinamen.

```
#!/bin/bash
# Linux CI – Beispielcode, MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
# Ein einfaches Skript mit Variable und Schleife

NAME="$1"

if [ -z "$NAME" ]; then
  NAME="Unbekannt"
fi

echo "Hallo, $NAME!"
echo "Heute ist $(date +%A)."

echo "Ich zähle bis 3:"
for i in 1 2 3; do
  echo "  -> $i"
done
```

> Der Wochentag kommt von `date +%A`. Die Sprache der Ausgabe hängt von der Runner-Umgebung ab und ist auf GitHub-Runnern in der Regel Englisch.

Datei anlegen: `.github/workflows/04-skript.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 04 - Mein erstes Skript

on:
  workflow_dispatch:
    inputs:
      name:
        description: "Wie heißt du?"
        required: true
        default: "Welt"
        type: string
      fehlerfall:
        description: "Fehlendes Ausführungsrecht testen"
        required: true
        type: boolean
        default: false

env:
  KURS: "Linux lernen"

jobs:
  skript:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Variablen anzeigen
        run: |
          echo "Kurs: $KURS"
          echo "Gestartet von: $GITHUB_ACTOR"
          echo "Repository: $GITHUB_REPOSITORY"

      - name: Rechte ansehen
        run: ls -l skripte/begruessung.sh

      - name: Skript ausführbar machen
        if: ${{ !inputs.fehlerfall }}
```

```
    run: chmod +x skripte/begruessung.sh

  - name: Rechte erneut ansehen
    if: ${{ !inputs.fehlerfall }}
    run: ls -l skripte/begruessung.sh

  - name: Ausführungsrecht für Fehlerprobe entfernen
    if: ${{ inputs.fehlerfall }}
    run: chmod -x skripte/begruessung.sh

  - name: Fehlendes Ausführungsrecht testen (absichtlicher Fehler)
    if: ${{ inputs.fehlerfall }}
    continue-on-error: true
    env:
      NAME: ${{ inputs.name }}
    run: ./skripte/begruessung.sh "$NAME"

  - name: Skript ausführen
    if: ${{ !inputs.fehlerfall }}
    env:
      NAME: ${{ inputs.name }}
    run: ./skripte/begruessung.sh "$NAME"
```

Warum `env:` statt direkt `${{ inputs.name }}` im Befehl?

GitHub setzt `${{ ... }}` ein, **bevor** die Shell den Befehl sieht. Würde der Eingabewert direkt in den Shell-Code eingesetzt, könnten Sonderzeichen den Befehl verändern. Bei der Übergabe über `env:` wird der Wert dagegen nicht in den Shell-Code eingesetzt. Die Shell liest ihn aus der Umgebungsvariable; durch die Anführungszeichen in `"$NAME"` wird er als **ein einzelnes Argument** an das Skript übergeben. Das schützt diese Übergabe vor Shell-Code-Einschleusung und Wortaufteilung; andere Verwendungen des Werts müssen weiterhin passend geprüft werden.

Die Faustregel: **Setze nicht vertrauenswürdige Eingaben nicht direkt in den Shell-Code von `run:` ein. Übergib sie über `env:` und verwende sie als quotierte Variable, zum Beispiel `"$NAME"`.** (`GITHUB_ACTOR` und `GITHUB_REPOSITORY` stellt GitHub bereits als Umgebungsvariablen bereit.)

Zwei Arten von Variablen, nicht verwechseln

Schreibweise	Wer wertet aus?	Beispiel
`$NAME`	Die Linux-Shell zur Laufzeit	`echo "$NAME"`
`${{ ... }}`	GitHub Actions, vor dem Start des Befehls	`env: NAME: ${{ inputs.name }}`

Rechte lesen: `-rwxr-xr-x`

```
-  rwx  r-x  r-x
|   |    |    └─ alle anderen: lesen, ausführen
|   |    └────── Gruppe: lesen, ausführen
|   └─────────── Besitzer: lesen, schreiben, ausführen
└─────────────── Dateityp (- = Datei, d = Ordner)
```

`chmod +x` fügt das **x** (ausführbar) hinzu. Ohne dieses Recht schlägt `./skript.sh` meist mit `Permission denied` fehl. Alternativ ginge `bash skripte/begruessung.sh`; dafür braucht die Datei kein x-Recht.

Kommandos und Konzepte

Element	Zweck
`#!/bin/bash`	Erste Zeile: „Führe mich mit Bash aus“
`NAME="Anna"`	Variable setzen (**keine Leerzeichen** um `=`)
`$NAME`	Variable lesen
`$1`	Erster Parameter beim Aufruf
`$(date)`	Ergebnis eines Kommandos einsetzen
`if [ ... ]; then ... fi`	Bedingung
`for i in ...; do ... done`	Schleife
`chmod +x`	Datei ausführbar machen

Ausprobieren und kontrollieren

1. Starte **„04 - Mein erstes Skript“**. Trage im Feld `name` einen Namen ein und wähle **Run workflow**.

2. Öffne den Job `skript`. Prüfe in den Schritten „Rechte ansehen“ und „Rechte erneut ansehen“, dass nach `chmod +x` ein `x` in der Rechteanzeige steht.
3. Klappe „Skript ausführen“ auf. Erwartet werden eine Begrüßung mit deinem Namen, ein Wochentag und die Zahlen 1 bis 3.

Aufgaben

1. Starte den Workflow mit einem Namen, der Leerzeichen enthält, zum Beispiel `Ada Lovelace`. Weshalb kommt der vollständige Name als ein Parameter beim Skript an?
2. Erstelle über die Branch-Auswahl auf dem Tab **Code** einen Arbeitsbranch vom Standardbranch, wie in der Vorbereitung beschrieben. Ändere darin die Schleife in `skripte/begruessung.sh`, sodass sie bis 5 zählt. Committe und pushe die Änderung. Starte den Workflow mit diesem Branch und prüfe die fünf ausgegebenen Zahlen.
3. Ergänze im Skript die Uhrzeit mit `date +%H:%M`. Committe und pushe die Änderung auf den Arbeitsbranch. Die konkrete Uhrzeit hängt vom Ausführungszeitpunkt und der Umgebung ab.
4. Committe und pushe die Workflow-Änderung auf den Branch. Starte den Workflow erneut und setze die Eingabe `fehlerfall` (Beschreibung: `Fehlendes Ausführungsrecht testen`) auf `true`. Der Workflow entfernt mit `chmod -x` das Ausführungsrecht und versucht das Skript direkt auszuführen. Prüfe den erwarteten Fehler `Permission denied`; `continue-on-error: true` lässt den übrigen Job weiterlaufen. Das genaue Verhalten im GitHub-Runner wurde noch nicht getestet.

Häufige Anfängerfehler

Fehler	Ursache
Workflow-Dateien sind unter **Actions** nicht sichtbar	Sie müssen unter `.github/workflows/` liegen und auf dem Standardbranch vorhanden sein
`mapping values are not allowed`	YAML-Einrückung falsch (nur **Leerzeichen**, keine Tabs)
`No such file or directory`	`actions/checkout` vergessen oder Tippfehler im Pfad
`Permission denied`	`chmod +x` fehlt

Fehler	Ursache
`command not found`	Tippfehler im Kommando
`grep`-Schritt fehlgeschlagen	Kein Treffer ergibt Exit-Code 1; bei einer absichtlichen Fehlerprobe kann das erwartet sein
`File exists` bei `mkdir`	Ordner gibt es schon; `mkdir -p` verwenden

Spickzettel (`CHEATSHEET.md`)

```
pwd              Wo bin ich?
ls -la           Dateien anzeigen
cat datei        Datei lesen
mkdir -p ordner  Ordner anlegen
cp / mv          Kopieren / Verschieben
rm datei         Ohne Papierkorb löschen
head / tail      Anfang / Ende
wc -l            Zeilen zählen
grep "x" datei   Suchen (kein Treffer = Exit-Code 1)
sort             Sortieren
|                Pipe
> / >>           In Datei schreiben / anhängen
chmod +x         Ausführbar machen
$VAR             Variable lesen
env: in YAML     Eingaben an die Shell übergeben
```